SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Student 25 **PATAN MOHAMMED RIYAZ KHAN** 192572126RC 4 / 4 submitted

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q1. Intelligent Customer Feedback Analysis

CODE

```
import re

POSITIVE_KEYWORDS = {
    "good", "great", "excellent", "amazing", "love", "wonderful",
    "fantastic", "happy", "satisfied", "best", "awesome", "perfect"
}

NEGATIVE_KEYWORDS = {
    "bad", "terrible", "awful", "hate", "worst", "poor",
    "horrible", "disappointed", "disappointing", "slow", "broken", "useless"
}

def preprocess(feedback: str) -> str:
    feedback = feedback.lower().strip()
    feedback = re.sub(r"[^a-z\s]", " ", feedback)
    feedback = re.sub(r"\s+", " ", feedback).strip()
    return feedback

def count_keywords(words, keyword_set) -> int:
    count = 0
    for w in words:
        if w in keyword_set:
            count += 1
    return count

def classify_sentiment(pos_count: int, neg_count: int) -> str:
    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    return "Neutral"

def analyse_feedback(feedback: str):
    cleaned = preprocess(feedback)
    words = cleaned.split()
    pos_count = count_keywords(words, POSITIVE_KEYWORDS)
    neg_count = count_keywords(words, NEGATIVE_KEYWORDS)
    sentiment = classify_sentiment(pos_count, neg_count)
    return pos_count, neg_count, sentiment

def main():
    feedbacks = [
        "The product is GREAT and I absolutely love the amazing quality!",
        "Terrible experience. The item was broken and customer service was awful.",
        "It was okay, nothing special. Not bad, not great.",
        "Excellent service! The best purchase I have ever made. Fantastic delivery!",
        "Horrible product. Worst quality ever. Very disappointed and slow delivery."
    ]

    print("=" * 60)
    print(" INTELLIGENT CUSTOMER FEEDBACK ANALYSIS SYSTEM ")
    print("=" * 60)

    for i, feedback in enumerate(feedbacks, 1):
        pos, neg, sentiment = analyse_feedback(feedback)
        print(f"\nFeedback #{i}: {feedback}")
        print(f"Positive Keywords Found : {pos}")
        print(f"Negative Keywords Found : {neg}")
        print(f"Sentiment Classification: {sentiment}")
        print("-" * 60)

if __name__ == "__main__":
    main()
```

OUTPUT

```
============================================================
 INTELLIGENT CUSTOMER FEEDBACK ANALYSIS SYSTEM
============================================================
```

```
Feedback #1: The product is GREAT and I absolutely love the amazing quality!
Positive Keywords Found : 3
Negative Keywords Found : 0
Sentiment Classification: Positive
------------------------------------------------------------

Feedback #2: Terrible experience. The item was broken and customer service was awful.
Positive Keywords Found : 0
Negative Keywords Found : 3
Sentiment Classification: Negative
------------------------------------------------------------

Feedback #3: It was okay, nothing special. Not bad, not great.
Positive Keywords Found : 1
Negative Keywords Found : 1
Sentiment Classification: Neutral
------------------------------------------------------------

Feedback #4: Excellent service! The best purchase I have ever made. Fantastic delivery!
Positive Keywords Found : 3
Negative Keywords Found : 0
Sentiment Classification: Positive
------------------------------------------------------------

Feedback #5: Horrible product. Worst quality ever. Very disappointed and slow delivery.
Positive Keywords Found : 0
Negative Keywords Found : 4
Sentiment Classification: Negative
------------------------------------------------------------
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q2. Comprehensive Password Strength Checker

CODE

```
def analyze_password(password: str) → dict:
    info = {
        "length": len(password),
        "has_upper": False,
        "has_lower": False,
        "has_digit": False,
        "has_special": False,
        "upper_count": 0,
        "lower_count": 0,
        "digit_count": 0,
        "special_count": 0,
    }

    for ch in password:
        if "A" ≤ ch ≤ "Z":
            info["has_upper"] = True
            info["upper_count"] += 1
        elif "a" ≤ ch ≤ "z":
            info["has_lower"] = True
            info["lower_count"] += 1
        elif "0" ≤ ch ≤ "9":
            info["has_digit"] = True
            info["digit_count"] += 1
        else:
            info["has_special"] = True
            info["special_count"] += 1

    return info


def classify_strength(info: dict) → str:
    length = info["length"]
    category_count = (
        int(info["has_upper"])
        + int(info["has_lower"])
        + int(info["has_digit"])
        + int(info["has_special"])
    )

    if length == 0:
        return "Weak"
    if length < 6 or category_count ≤ 1:
        return "Weak"
    if (6 ≤ length < 10) or category_count == 2:
        return "Moderate"
    if length ≥ 12 and category_count == 4:
        return "Very Strong"
    return "Strong"


def get_feedback(password: str, info: dict) → list:
    feedback = []

    if info["length"] == 0:
        return ["Password cannot be empty."]

    if info["length"] < 8:
        feedback.append("Increase length to at least 8 characters.")
    elif info["length"] < 12:
        feedback.append("Try increasing length (12+ is better).")

    missing = []
    if not info["has_upper"]:
        missing.append("uppercase letter")
    if not info["has_lower"]:
        missing.append("lowercase letter")
    if not info["has_digit"]:
```

```
        missing.append("digit")
    if not info["has_special"]:
        missing.append("special symbol")

    if missing:
        feedback.append("Add: " + ", ".join(missing) + ".")

    # Simple repetition check
    if info["length"] ≥ 4:
        first = password[0]
        all_same = True
        for ch in password:
            if ch ≠ first:
                all_same = False
                break
        if all_same:
            feedback.append("Avoid repeating the same character (e.g., 'aaaaaa').")

    return feedback


def password_strength_checker(password: str) → None:
    info = analyze_password(password)
    strength = classify_strength(info)
    feedback = get_feedback(password, info)

    print("Password Strength:", strength)
    print("Details:")
    print(f"  Length: {info['length']}")
    print(f"  Uppercase: {info['upper_count']}")
    print(f"  Lowercase: {info['lower_count']}")
    print(f"  Digits: {info['digit_count']}")
    print(f"  Symbols: {info['special_count']}")

    print("Feedback:")
    if not feedback:
        print("  Looks good! Keep it secure.")
    else:
        for item in feedback:
            print(" ", f"- {item}")


if __name__ == "__main__":
    s = input().rstrip("\n")
    password_strength_checker(s)
```

INPUT	acd@1234
OUTPUT	(no output)

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q3. Structured Server Log Error Analyser

CODE

```
def read_logs():
    """
    You can replace this function to read from a file if needed.
    For now, it reads logs from user input line-by-line until 'END'.
    """
    logs = []
    print("Enter log lines (type END to finish):")
    while True:
        line = input()
        if line.strip().upper() == "END":
            break
        logs.append(line)
    return logs


def extract_error_lines(log_lines):
    """
    Extract only lines related to errors using string searching.
    Here we treat any line containing 'ERROR' as an error line.
    """
    error_lines = []
    for line in log_lines:
        if "ERROR" in line or line.startswith("ERROR") or line.find("ERROR") != -1:
            error_lines.append(line)
    return error_lines


def categorize_error(line):
    """
    Categorize error type based on keywords inside the line.
    Modify keywords as per your log format.
    """
    keywords = {
        "Database Error": ["DB", "DATABASE", "sql", "SQL", "db error", "constraint"],
        "404 Not Found": ["404", "not found", "Not Found"],
        "Authentication Error": ["AUTH", "AUTHENTICATION", "login failed", "unauthorized"],
        "Timeout Error": ["timeout", "TIMEOUT"],
        "Resource Error": ["resource", "out of memory", "OOM"],
        "General Error": ["ERROR"]
    }

    lower = line.lower()
    for category, keys in keywords.items():
        for k in keys:
            if k.lower() in lower:
                return category
    return "General Error"


def remove_duplicates(lines):
    """
    Remove duplicate error messages for concise reporting.
    We consider duplicates based on the full line text.
    """
    unique = []
    seen = set()
    for line in lines:
        if line not in seen:
            unique.append(line)
            seen.add(line)
    return unique


def summarize_errors(unique_error_lines):
    """
    Count occurrences of categorized error types.
    Also returns a formatted summary string.
```

```
    """
    counts = {}
    for line in unique_error_lines:
        category = categorize_error(line)
        counts[category] = counts.get(category, 0) + 1

    # Sort by count descending for better reporting
    sorted_counts = sorted(counts.items(), key=lambda x: x[1], reverse=True)

    return sorted_counts


def generate_report(log_lines):
    error_lines = extract_error_lines(log_lines)
    unique_error_lines = remove_duplicates(error_lines)
    summary = summarize_errors(unique_error_lines)

    report_lines = []
    report_lines.append("=== Error Report ===")
    report_lines.append(f"Total Error Lines (after duplicate removal): {len(unique_error_lines)}")
    report_lines.append("Error Types Count:")

    if not summary:
        report_lines.append("No errors found.")
    else:
        for category, cnt in summary:
            report_lines.append(f"- {category}: {cnt}")

    report_lines.append("\nUnique Error Messages:")
    if unique_error_lines:
        for msg in unique_error_lines:
            report_lines.append(f"* {msg}")

    return "\n".join(report_lines)


def main():
    logs = read_logs()
    print()
    print(generate_report(logs))


if __name__ == "__main__":
    main()
```

INPUT

```
INFO Starting server...
WARN CPU usage high
ERROR Database connection failed: constraint violation
ERROR Database connection failed: constraint violation
ERROR 404 Not Found: /api/user/10
INFO Request processed
ERROR Authentication failed: unauthorized access attempt
ERROR Timeout while waiting for response
WARN Disk space low
ERROR Resource out of memory (OOM)
END
```

OUTPUT

```
(no output)
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: Scenario Based Program Writing • Student Submissions Report

Q4. Email Domain Frequency Analyser

CODE

```
def extract_domain(email):
    """
    Returns domain part of email if valid, otherwise returns None.
    A "valid enough" email for this task:
    - contains exactly one '@'
    - has non-empty domain part after '@'
    """
    if not isinstance(email, str):
        return None

    email = email.strip()
    if email.count('@') != 1:
        return None

    local, domain = email.split('@')
    if not local.strip() or not domain.strip():
        return None

    # Basic check to avoid spaces in domain
    if ' ' in domain:
        return None

    return domain.lower()


def domain_frequency_analysis(emails):
    domain_count = {}

    for email in emails:
        domain = extract_domain(email)
        if domain is None:
            continue  # skip malformed emails gracefully

        if domain in domain_count:
            domain_count[domain] += 1
        else:
            domain_count[domain] = 1

    # Identify the most common domain
    max_domain = None
    max_freq = 0

    for domain, freq in domain_count.items():
        if freq > max_freq:
            max_freq = freq
            max_domain = domain

    return domain_count, max_domain, max_freq


def main():
    # Example input (replace with your own dataset)
    emails = [
        "mohammed@gmail.com",
        "alice@yahoo.com",
        "bob@gmail.com",
        "charlie@outlook.com",
        "invalidemail.com",
        "noatsign.com",
        "dave@gmail.com",
        "   eve@YAHOO.com  ",
        "frank@",
        "@gmail.com"
    ]

    domain_count, max_domain, max_freq = domain_frequency_analysis(emails)
```

```
    print("Domain Frequencies:")
    if not domain_count:
        print("No valid email domains found.")
        return

    for domain, freq in sorted(domain_count.items(), key=lambda x: x[0]):
        print(f"{domain}: {freq}")

    print("\nMost Common Domain:")
    print(f"{max_domain} ({max_freq} times)")


if __name__ == "__main__":
    main()
```

OUTPUT

```
Domain Frequencies:
gmail.com: 3
outlook.com: 1
yahoo.com: 2

Most Common Domain:
gmail.com (3 times)
```